



نظري

5



6



720 sp

كلية الهندسة المعلوماتية

السنة الأولى

Object-Oriented Programming (OOP)

د. محمد الأحمد / د. عبد الرحمن اللحام



RB Informatics; 07/05/2025

البرمجة 2

إن البرمجة كائنية التوجه (OOP) تستخدم مجموعة من الأساليب لتحقيق:

1. إعادة الاستخدام (Reusability): إعادة استخدام الكود بدلاً من كتابته من جديد.
2. المرونة (Adaptability): القدرة على التعديل بسهولة.

أهم تلك الأساليب:

1. التجريد (Abstraction).
2. التغليف (Encapsulation).
3. الوراثة (inheritance).
4. تعدد الأشكال (Polymorphism).

تعريف تركيب الكائنات (Object composition):

- يمكن أن تتكون ال objects من objects أخرى.
- يمكن أن تكون ال objects جزءاً من objects أخرى.
- تعرف العلاقة بين ال objects باسم "التجميع" (Aggregation).

الخاصية	COMPOSITION	AGGREGATION
الاعتمادية	لا يمكن أن يعيش ال object بدون مكوناته	يمكن أن يعيش ال object بدون ال object الآخر.
مثال	السيارة تمتلك محرك (المحرك ضروري)	طالب ينتمي إلى كلية (يمكن أن يوجد بدونها)

ماهي الوراثة (Inheritance)?

- قد تكون هناك سمات مشتركة بين عدة classes مختلفة.
- نعرّف الخصائص المشتركة في class أب (Superclass).
- ال classes الأبناء (Subclasses) تستخدم الوراثة كي تمتلك نفس الخصائص (Instance variables OR Fields) الموجودة لدى الآباء.

استخدام علاقة "Is-a-Kind-of"

لدينا طريقتان لتمييز ال class الفرعي عن ال class الرئيسي الذي يرث منه:

الطريقة الأولى:

إضافة functions جديدة إلى ال class الفرعي

الطريقة الثانية:

تغيير سلوك function موجود في ال class الأب، وهذا يعرف بال "overriding".

مقارنة بين علاقتي "Is-a" و "Is-like-a"

Is-a

يتم تجاوز (override) functions ال class الأب فقط.

النوع المشتق (Derived type) هو تماماً من نفس نوع ال class الأساسي، لأنه يملك نفس الواجهة (interface).

مثال: "الدائرة هي شكل"

(Circle is a Shape)

Is-like-a

يتم إضافة عناصر جديدة إلى الواجهة في النوع المشتق.

أي أنه يتم توسيع الواجهة (interface) وإنشاء نوع جديد (New type).

مثال: "مضخة الحرارة تشبه مكيف الهواء"

(Heat pump is-like-an Air Conditioner)

تعدد الأشكال Polymorphism

يشير مصطلح ال polymorphism إلى:

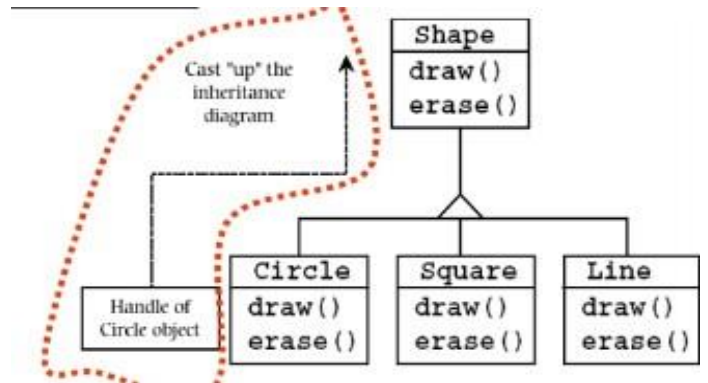
1. أشكال متعددة لنفس العملية، أي أن نفس العملية يمكن تنفيذها بطرق مختلفة بحسب ال object.
2. القدرة على طلب نفس العملية، لكن تنفذ بشكل مختلف حسب نوع ال object الذي استدعيت عليه.
3. يعتمد على مبدأ الوراثة (inheritance) واستبدال الكائنات (Object Substitution).

من أهم ما يمكننا فعله عند وجود عائلة من ال classes هو أن نتعامل مع object من class مشتق وكأنه object من ال class الأساسي. وهذا مهم جداً، لأن هذا يعني أنه يمكنك كتابة كود واحد يتجاهل التفاصيل الدقيقة للنوع ويتعامل فقط مع ال class الأب.

```
void doStuff(Shape s)
{
    s.erase();
    // ...
    s.draw();
}
```

```
Circle c = new Circle();
Triangle t = new Triangle();
Line l = new Line();

doStuff(c);
doStuff(t);
doStuff(l);
```



تخصيص الوراثة (Inheritance) في لغة Java:

ال inheritance تتحقق عند تحديد الكلاس الأب (super class) الذي يرث منه الكلاس الإبن (subclass) وذلك باستخدام كلمة extends.

مثال:

```
public class InventoryItem{
    private float price;
    private String condition;
}
```

```
public class Movie extends InventoryItem{
    private String title;
    private int Length;
}
```

إن Movie يرث جميع ال variables و ال methods الخاصة ب InventoryItem. وبالتالي فإن Movie يمتلك حالياً Length, title, condition, price ... (نلاحظ أن ال private variables تورث).

ماذا يحدث عند إنشاء object ل subclass ؟

مثال:

```
Movie movie1 = new Movie();
```

إذا لم يكن يوجد constructors معرفة مسبقاً:

أولاً، ال constructor الافتراضي (no-arg constructor) يُستدعى في ال superclass، ثم يُستدعى (no-arg constructor) في subclass.

المرجعية super:

تشير إلى ال class الأساسي.
مفيد في استدعاء ال constructors الخاصة بال classes الأساسية.
يجب أن تكون السطر الأول في ال derived class constructor (derived تعني مشتق).
يمكن أن تستخدم في استدعاء أي method لأي class أساسي.

مثال:

```
public class InventoryItem {
    InventoryItem(String cond){
        System.out.println("InventoryItem");
        ...
    }
}
class Movie extends InventoryItem{
    Movie (String title){
        super(title);
        ....
        System.out.println("Movie");
    }
}
```

استخدام Superclass constructors:

نستخدم super() لاستدعاء superclass constructor.

```
public class InventoryItem{
    InventoryItem (float p,String cond){
        price = p;
        condition = cond;
    }
}
```

```
public class Movie extends InventoryItem{
    Movie (String t, float p, String cond){
```

```
super (p, cond);
title = t;
}
}
```

تجاوز ال superclass methods (overriding)

كما قلنا سابقاً، أن ال subclass يرث جميع ال methods الخاصة بال superclass. وبما أنه يستطيع ذلك، فإنه يستطيع أيضاً تجاوز ال methods بطريقته الخاصة (أي أنه يستخدم نفس ال method ويغير مضمونها)

■ ملاحظة: إن ال subclass method يجب أن تمتلك نفس توقيع ودلالات ال superclass method الذي ورثته.

مثال:

```
public class Vcr extends InventoryItem {
    public float calcDeposit(int custId){
        if...
        return itemDeposit;
    }
}
```

استدعاء superclass methods

إذا قام ال subclass بتجاوز methods، يبقى بإمكانه استدعاء ال superclass method الأساسي. لاستدعاء superclass method من ال subclass نستخدم "super.method()".

```
public class Vcr extends InventoryItem{
    public float calcDeposit(int custId){
        itemDeposit = super.calcDeposit (custId);
        return (itemDeposit + vcrDeposit);
    }
}
```

استخدام العامل instanceof

يمكننا تحديد النوع الصحيح ل object عن طريق استخدام العامل instanceof. مرجعية object يمكن أن تخفض للنوع الصحيح إذا لزم الأمر.

```
public void aMethod (InventoryItem i){
    ...
}
```

```
if (i instanceof Vcr)
    ((Vcr)i).playTestTape();
}
```

the end

